



Departamento de Programación
Facultad de Informática
Universidad Nacional del Comahue



Programación Concurrente



Sincronización de cooperación
Monitores

Sincronización competencia y cooperación

Mecanismos

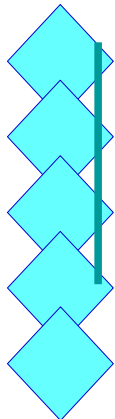
```
graph TD; A[Mecanismos] --> B[Monitores + esperas guardadas]; A --> C[Semáforos binarios y generales]; A --> D[Cerros + Variables de condición]; A --> E[Otras posibilidades ...];
```

Monitores + esperas guardadas

Semáforos binarios y generales

Cerros + Variables de condición

Otras posibilidades ...



Sincronización - Monitores

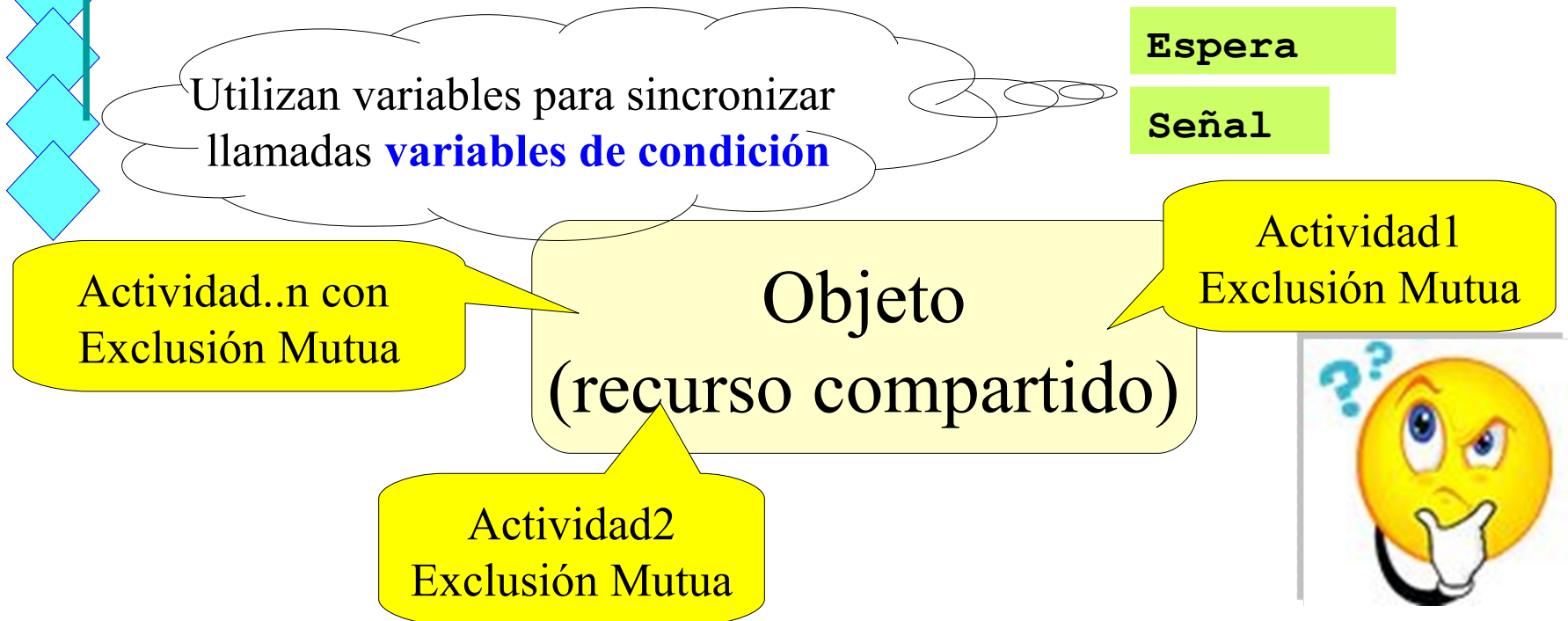


- Un monitor es un mecanismo de abstracción de datos que se aplica en un ambiente concurrente
- Un monitor asegura exclusión mutua
- Un monitor es un objeto pasivo, un recurso compartido
- Un proceso que invoca un método sobre un monitor puede ignorar cómo el método es implementado. Todo lo que importa son los efectos visibles de la invocación
- El programador de un monitor puede ignorar cómo y dónde los métodos del monitor son usados.

Sincronización - Monitores

- Mecanismo de sincronización de **alto nivel**
 - garantiza que solamente pueda existir un hilo activo dentro de la sección crítica.
- Un monitor **permite suspender un hilo dentro de la sección crítica** posibilitando que otro hilo pueda acceder a la misma.
- Este segundo hilo puede abandonar el monitor, liberándolo, o suspenderse dentro del monitor.
- El hilo original se despierta y continúa su ejecución dentro del monitor.
- Este esquema es escalable a múltiples hilos
(varios hilos pueden suspenderse dentro de un monitor).

Sincronización - Monitor

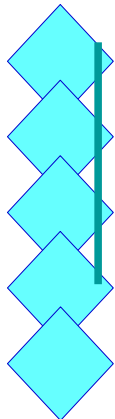


- Las actividades (operaciones) van encapsulados dentro de un módulo.
- Dentro de cada **módulo** sólo un proceso puede estar activo cada vez
- Si el monitor se ha codificado correctamente no puede ser utilizado incorrectamente por ningún proceso que desee usar el recurso.

Sincronización - Monitores



- En Java ... la semántica de monitor se logra por la combinación de “synchronized” y los métodos:
wait(), notify(); notifyAll()
- Cada objeto tiene un *lock* y un *conjunto de espera* (CE)
- Cualquier objeto puede servir como *monitor*. Aunque cada lenguaje define sus propios detalles
- Cada CE mantiene los hilos bloqueados por un *wait()* sobre el objeto
- El CE interactúa con el lock, entonces *wait()*, *notify()* y *notifyAll()* requieren sincronización.



Sincronización - Monitores



Esperas guardadas (dependen de una condición)

- En general las invocaciones de `wait()` se hacen dentro de un bucle `while`.
- Cuando una acción es reasumida, la tarea en espera NO SABE si la condición por la que estaba esperando es verdadera.
- Solo sabe que lo liberaron, y debe volver a verificar, para mantener la propiedad de seguridad.
- Es una buena práctica que este estilo se utilice aun cuando haya una única instancia que pueda esperar por la condición.

Sincronización - Monitor

- No es posible señalar a un hilo en especial.
 - Es aconsejable bloquear a los hilos en el CE con una condición de guarda en conjunción con notifyAll().

while (!condicion)

try { ... wait(); }

catch (InterruptedException e) { ... }

- Todos son despertados, comprobarán la condición y volverán a bloquearse excepto los que la encuentren verdadera .
- **No es aconsejable** comprobar la condición de guarda con **if**
- Los **campos protegidos** por el monitor deben declararse **private**

Semántica del monitor

Cuando un método `synchronized` del monitor ejecuta un `wait()` libera la exclusión mutua sobre el monitor y el hilo corriente es ubicado en el CE del objeto.

Cuando desde otro método del monitor se ejecuta un `notify()`, un hilo del CE pasará al conjunto de hilos que esperan el cerrojo y se reanudará cuando sea planificado.

Cuando desde otro método del monitor se ejecuta `notifyAll()`, todos los hilos del CE pasarán al conjunto de hilos que esperan el cerrojo y se reanudarán cuando sean planificados.

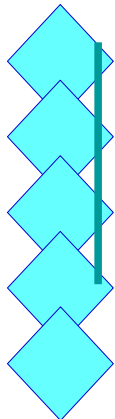
Cómo utilizar el método *wait()*

- Utilizar dentro de un método *synchronized* y dentro de un ciclo indefinido que verifica la condición:

```
synchronized void hacerCondicion() {  
    while (!Condicion)  
        this.wait();  
}
```

- Al ejecutar un **wait** sobre un objeto
 - El hilo se suspende, y es ubicado por la JVM en el CE del objeto.
 - Libera el lock** (de sincronización) del objeto.
 - Si el hilo mantiene otros locks en su poder, NO los libera!!!

Cuidado!!



Cómo usar el método *notify()*

- Utilizar dentro de un método *synchronized* :

```
synchronized void cambiaCondicion() {  
    ...//algo cambia en el estado del objeto  
    this.notifyAll() ;  
}
```

- Muchos thread pueden estar esperando sobre el objeto:
 - Con **notify()** solo un thread (no se sabe cual) es despertado.
 - Con **notifyAll()** todos los thread son despertados y cada uno decide si la notificación le afecta, o si no, vuelve a ejecutar el wait().
(podría suceder que la condición que espera aun no se cumpla)

Cómo usar el método *notify()*

- El hilo *H* suspendido (por efecto de un `wait()`) **SIEMPRE debe esperar (al menos)** hasta que el hilo que invoca ***notify()*** o ***notifyAll()*** libere el lock del objeto.
- *H* debe volver a obtener el lock del objeto, entonces puede ocurrir que se bloquee si algún otro hilo lo obtiene primero.
- Cuando *H* logra obtener el lock retoma desde el punto de su `wait()`
- Si el CE está vacío `notify()` y `notifyAll()` no tienen efecto

Mecanica de monitores

Consideremos el siguiente ejemplo inutil

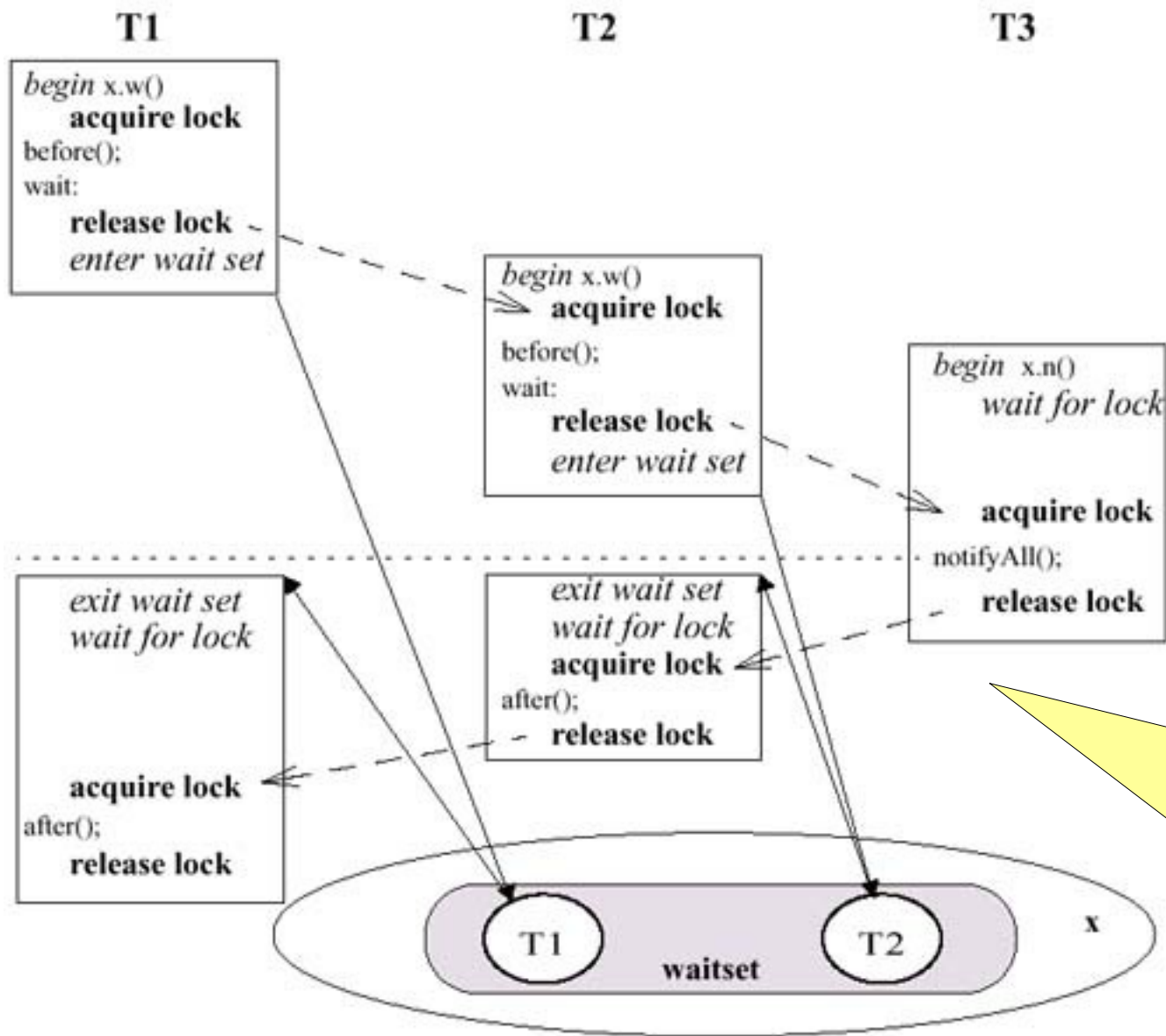
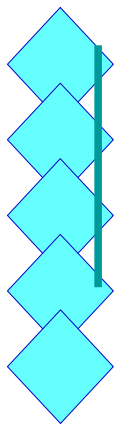
```
class MonX {
    synchronized void w() throws InterruptedException {
        this.before();
        this.wait();
        this.after();
    }

    synchronized void n() {
        notifyAll();
    }

    void before() {}
    void after() {}
}
```

Y consideremos 3 hilos que invocan los métodos del monitor MonX sobre una instancia compartida x, (x instancia de MonX)

Ejemplo tomado de: *“Concurrent Programming in Java” - Doug Lea*



Para que todo funcione, ¿importa el orden de ejecución de los hilos T1, T2 y T3?